

STEMVERSE_AGENT_PROTOCOL.md

Version: 1.0

Purpose: Unified operating instructions for all AI coding agents working on STEMVerse.

Applies To:

- Opus 4.6
- Kimi K2.6
- Gemini 3.5
- DeepSeek V4
- GLM 5.1
- Claude Code
- Cursor Agents
- RooCode
- OpenCode
- Cline
- Windsurf

PROJECT STATUS

STEMVerse is a large-scale electronics simulation platform.

Current State:

- Runtime Engine Established
- Pixi Renderer Established
- Workspace Runtime Established
- Breadboard Runtime Established
- Electrical Connectivity Established
- Signal Propagation Established
- Interactive Placement Established

Project Scale:

- 500,000+ tests
- 70+ test files
- Multi-package monorepo

This project is in active development.

GLOBAL RULES

RULE 1

Do NOT redesign architecture.

Reuse existing architecture.

RULE 2

Do NOT replace existing systems.

Extend existing systems.

RULE 3

Do NOT rename public APIs.

Maintain backward compatibility.

RULE 4

Follow existing patterns.

Reuse:

- Registries
 - Synchronizers
 - Validators
 - Renderer Adapters
 - Snapshot Systems
 - Serialization Systems
 - Lifecycle Systems
-

RULE 5

Validation must be:

- warning-only

- console.warn
 - never throw
-

RULE 6

Deep-copy safety is mandatory.

Use clone-safe structures.

RULE 7

All additions must support:

- snapshot integration
 - serialization
 - import/export
 - lifecycle cleanup
-

RUNTIME STANDARDS

Every runtime registry must support:

- register
- get
- getAll
- update
- remove
- clear
- getKeys
- has

Use:

- Map storage
 - deterministic ordering arrays
-

LIFECYCLE STANDARDS

Every feature must integrate with:

- initialize()
 - stop()
 - reset()
 - destroy()
-

SERIALIZATION STANDARDS

Every feature must integrate with:

- getStageSnapshot()
 - exportProject()
 - importProject()
-

TESTING STANDARDS

Required tests:

- CRUD tests
- clone tests
- serialization tests
- snapshot tests
- renderer isolation tests
- lifecycle tests

Never mark a phase complete unless:

`pnpm --filter @stemverse/runtime-engine test`

passes

AND

`pnpm --filter @stemverse/runtime-engine build`

passes

DOCUMENTATION STANDARDS

Always update:

- progress.md
- MASTER_HANDOFF.md

Required:

- roadmap updates
 - metrics updates
 - changelog updates
-

GIT WORKFLOW

After successful implementation:

`git status`

`git add .`

`git commit -m "Phase XX complete"`

`git push origin current-branch`

Create tag:

`git tag -a phase-XX-stable -m "Phase XX stable"`

`git push origin phase-XX-stable`

MODEL RESPONSIBILITIES

OPUS 4.6

Primary Role:

Architecture Lead

Responsibilities:

- architecture review
- large refactors
- integration planning
- dependency analysis
- final verification

Priority:

Correctness over speed.

KIMI K2.6

Primary Role:

Implementation Lead

Responsibilities:

- feature implementation
- runtime integration
- TypeScript development
- Pixi integration
- React integration

Priority:

Follow existing patterns exactly.

GEMINI 3.5

Primary Role:

Rapid Feature Builder

Responsibilities:

- medium-size implementations
- UI integration
- runtime extensions
- bug fixes

Priority:

Fast iteration.

DEEPSEEK V4

Primary Role:

Verification Engineer

Responsibilities:

- test generation
- edge-case coverage
- stress testing
- regression detection

Priority:

Maximum coverage.

GLM 5.1

Primary Role:

Maintenance Engineer

Responsibilities:

- documentation updates
- cleanup
- small fixes
- consistency checks
- handoff preparation

Priority:

Stability and clarity.

REQUIRED OUTPUT FORMAT

Every completed phase must report:

1. Files Modified

2. Files Created
3. Tests Passing Count
4. Build Status
5. progress.md Updates
6. MASTER_HANDOFF.md Updates
7. Remaining Risks
8. Concise Summary

Do not claim completion unless tests and build pass.

STEMVERSE PRINCIPLE

Follow the existing architecture.

Do not invent a new one.

Extend what already exists.

Consistency is more important than novelty.